GIT & GITHUB

COMPLETE REVISION NOTES + COMMAND CHEAT SHEET

A practical, beginner-friendly guide based on your Git exercises: repository setup, commits, version history, branches, merging, GitHub synchronization, .gitignore, and troubleshooting.

GOAL: Revise the concepts quickly, understand why each command is used, and recall one complete workflow from local project to GitHub.

Contents

- PART 1 Git & GitHub fundamentals
- PART 2 One complete Git workflow
- PART 3 Git versioning and previous commits
- PART 4 Branches and merging
- PART 5 Local and remote repositories
- PART 6 .gitignore and .DS_Store
- PART 7 Common errors and solutions
- PART 8 High-priority interview questions
- PART 9 Final command cheat sheet

This guide consolidates the topics and exercises from your terminal history and earlier practice. Commands are shown for macOS Terminal / VS Code terminal.

PART 1: Git & GitHub Fundamentals

1. What is Git?

Git is a free, distributed version control system that tracks changes to files and helps developers manage different versions of a project.

Example: If your latest Python code has an error, Git allows you to inspect an earlier version.

2. What is GitHub?

GitHub is an online platform that hosts Git repositories and allows developers to store, share, and collaborate on code.

Example: You manage your project using Git on your Mac and push it to GitHub to store it online.

3. Git vs. GitHub

Git	GitHub
Version control system	Online code hosting platform
Tracks changes and commits	Hosts repositories online
Works locally on your computer	Provides online access and collaboration
Command-line tool	Website and collaboration platform

4. Important Git terminology

Term	Simple meaning
Repository (repo)	A project managed by Git, including its history
Commit	A saved snapshot of your project
Branch	An independent line of development
main	The commonly used primary branch
Remote	A reference to another repository, such as one on GitHub
origin	The conventional name for your GitHub remote
Staging	Selecting changes for the next commit
Merge	Combining changes from one branch into another
Clone	Copying an existing remote repository to your computer
Push	Uploading commits to a remote repository
Pull	Downloading and integrating remote changes

Remember the three main steps

```
Working Directory
      | git add
      v
Staging Area
      | git commit
      v
Local Repository
      | git push
      v
GitHub Repository
```

Interview answer - What is version control? Version control tracks changes to files over time, allowing developers to review previous versions, compare changes, and recover earlier work.

PART 2: One Complete Git Workflow

Workflow A: Create a new project and upload it to GitHub

Step 1: Open the project in VS Code

Open the folder, then select Terminal -> New Terminal. Check your location, list files, and check the Git installation:

```
pwd
ls -la
git --version
```

Step 2: Initialize Git

```
git init
```

Run this once in the project folder. Git initializes a repository in that folder.

Step 3: Create .gitignore

Create a file named .gitignore inside the repository. Example:

```
.DS_Store
__pycache__/
*.pyc
.venv/
asi-venv/
.ipynb_checkpoints/
```

This prevents matching untracked files, such as macOS .DS_Store files and local Python environments, from being picked up by Git.

Step 4: Check your changes

```
git status
```

This displays untracked files and other changes.

Step 5: Stage files

```
git add .
git status
git diff --cached
```

git add . stages changes under the current directory. Check the staging area with git status. Review staged changes with git diff --cached.

Step 6: Create your first commit

```
git commit -m "Initial commit"
```

This saves a snapshot in your local Git history.

Step 7: Create a GitHub repository

Create an empty repository on GitHub and copy its URL. For the simplest initial setup, create it without a README, .gitignore, or license if you already created these locally.

```
https://github.com/USERNAME/REPOSITORY.git
```

Replace the example URL with your actual repository URL.

Step 8: Connect Git to GitHub

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git remote -v
git branch -M main
```

origin is the remote nickname. git remote -v checks the connection. git branch -M main renames the current branch to main.

Step 9: Push for the first time

```
git push -u origin main
```

This pushes your branch to GitHub and sets its upstream tracking branch. -u is the short form of --set-upstream.

After the upstream has been set, you can normally use the following for subsequent pushes from that branch:

```
git push
```

Remember: The initial push uses -u (or --set-upstream) so Git knows which remote branch your local branch tracks.

Workflow B: Clone an existing GitHub repository

When a repository already exists on GitHub, you normally clone it rather than initialize and connect another repository.

```
cd /path/to/parent-folder
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY
git status
```

git clone downloads the repository, its existing history, and its remote configuration.

Important: Do not run git init unnecessarily inside a repository you have just cloned.

Workflow C: Everyday Git workflow

This is the workflow to remember for daily development. When remote changes may exist, first check the state and pull/integrate the latest changes, then edit your files:

```
git status
git pull

# Edit and save files in VS Code
git status
git diff
git add .
git status
git diff --cached
git commit -m "Describe the changes"
git push
```

Step	Command / action	Purpose
1	git status	Check the current state
2	git pull	Get and integrate remote changes
3	Edit files	Make your changes
4	git diff	Review unstaged changes
5	git add .	Stage changes
6	git diff --cached	Review staged changes
7	git commit -m "message"	Save changes locally
8	git push	Upload commits to GitHub

When working alone, git pull may not be necessary every time, but it is useful when remote changes may exist. When collaborating, fetch and integrate remote changes before proceeding with your work.

PART 3: Git Versioning

You practised versioning by making multiple commits, checking out previous commits, and creating an experimental commit.

1. View commit history

```
git log
git log --oneline
```

Example history:

```
ee20816 modifed
b986c5f Add practice text
aafea8f practice
```

Each commit has a unique identifier and message.

2. View a particular commit

```
git show <commit_id>

# Example
git show b986c5f
```

This displays information about that commit and the changes it introduced.

3. Compare changes

```
git diff
git diff --cached
git diff <older_commit> <newer_commit>
```

Command	Purpose
git diff	Show unstaged changes
git diff --cached	Show staged changes
git diff <older_commit> <newer_commit>	Compare two commits

4. View an earlier version

```
git checkout <commit_id>

# Example
git checkout aafea8f
```

This checks out the files as they existed in that commit. Checking out a commit does not move your existing branch or update GitHub.

What is detached HEAD?

Normally, HEAD points to your current branch. When you check out a commit directly, HEAD points to that commit instead of a branch. This is called detached HEAD state.

You can inspect an old version and experiment, but new commits made in this state are not automatically attached to your existing branch.

Return to your original branch using either command:

```
git checkout demo_branch
# or
git switch demo_branch
```

Preserve experimental work

You created a commit named dd after checking out an older commit. You then created a branch to preserve that experiment. The general workflow is:

```
git switch -c experiment_branch
git push -u origin experiment_branch
```

Use the first command to create a branch at your current commit. Use the second if you also want to publish that branch to GitHub.

Interview answer - How does Git track versions? Git records commits in the repository history. Developers can view previous commits using git log, inspect individual commits using git show, and compare versions using git diff.

5. How can you undo changes?

Command	Purpose
git restore --staged <file>	Unstage a file while keeping its edits
git restore <file>	Discard unstaged edits to a file
git revert <commit_id>	Create a new commit that reverses an earlier commit
git reset	Normally unstages changes when used without a commit argument; other forms can move the branch and potentially discard work

Be careful: Commands that discard changes can cause lost work. Check git status first.

PART 4: Branches and Merging

You practised this in version_control_2 using branches such as gitbranch_1, gitbranch_2, and gitbranch_3.

1. What is a branch?

A branch is an independent line of development that allows you to work on changes without immediately changing another branch.

2. Essential branch commands

Command	Purpose
git branch	List local branches
git branch -a	List local and remote-tracking branches
git branch feature-name	Create a branch without switching to it
git switch main	Switch to main
git switch -c feature-name	Create and switch to a new branch
git merge feature-name	Merge the named branch into your current branch

You can also use git checkout <branch-name> to switch branches.

3. Complete branch workflow

Suppose you want to develop a feature separately from main.

```
git switch main
git pull
git switch -c feature-name

# Make changes, then save them
git add .
git commit -m "Add new feature"
git push -u origin feature-name

git switch main
git pull
git merge feature-name
git push
```

If main has no upstream configured, set it during the first push:

```
git push -u origin main
```

This is the direct branch-and-merge workflow you practised. On collaborative projects, a common alternative is to open a Pull Request on GitHub for review before merging.

Common interview questions

What is merging? Merging integrates changes from one branch into the branch currently checked out.

What is a merge conflict? A merge conflict occurs when Git cannot automatically reconcile conflicting changes. The developer resolves the affected files, stages them, and completes the merge.

What is a Pull Request? A Pull Request on GitHub proposes changes from one branch to another for review before merging.

PART 5: Local and Remote Repositories

1. What is origin?

origin is the conventional name for a remote repository, usually the GitHub repository connected to your local project.

```
git remote -v
git remote add origin <repository-url>
```

2. What is an upstream branch?

An upstream branch is the remote branch your local branch tracks by default for operations such as pushing and pulling.

```
Local branch:  main
     | tracks
     v
Remote branch: origin/main
```

3. Why does the first push sometimes fail?

You encountered this error:

```
fatal: The current branch experiment_branch has no upstream branch.
```

This means Git does not yet know the default remote branch for that local branch. Run:

```
git push -u origin experiment_branch
```

After successfully configuring the upstream, you can normally use git push. Each new local branch may need its own upstream configuration. Configuring main does not automatically configure every future branch.

4. Difference between push, pull, and fetch

Command	Meaning
git push	Uploads local commits to a remote repository
git pull	Fetches remote changes and integrates them into the current branch
git fetch	Downloads remote updates without integrating them into your current branch

Interview answer - Git vs. GitHub: Git manages version history. GitHub hosts Git repositories online and provides collaboration features such as Pull Requests.

PART 6: .gitignore and .DS_Store

1. What is .gitignore?

.gitignore contains file and directory patterns that Git should ignore when finding untracked files.

```
.DS_Store
__pycache__/
*.pyc
.venv/
asi-venv/
```

2. Why did .DS_Store appear?

macOS creates .DS_Store files to store folder-view settings. These files are hidden by default, so you may not notice them in Finder or VS Code.

```
ls -la
```

This lists hidden files in Terminal.

3. What if .DS_Store is already tracked?

Adding it to .gitignore does not automatically remove it from Git tracking. Use:

```
git rm --cached .DS_Store
git add .gitignore
git commit -m "Ignore DS_Store"
git push
```

This removes .DS_Store from Git tracking while keeping the local file on your Mac.

Interview answer - Why use .gitignore? We use .gitignore to prevent unnecessary or environment-specific files, such as caches and local configuration files, from being added to a repository.

PART 7: Common Errors You Encountered

Error or situation	Reason	Solution
git push says no upstream branch	The current branch has no configured tracking branch	git push -u origin <branch-name>
fatal: No configured push destination	No remote push destination is configured	Add origin, then push with -u
git pull says no tracking information	Git does not know which remote branch to integrate	Confirm the branch and set its upstream

Error or situation	Reason	Solution
pathspec 'v3.txt' did not match any files	The file was not in the current repository directory or the name was wrong	Check pwd and ls -la
nothing to commit, working tree clean	No new changes are staged or available to commit	Edit and save a file, then check git status
error: switch 'm' requires a value	No message was supplied after -m	git commit -m "Your message"
.DS_Store appears as untracked	macOS created a hidden file	Add it to .gitignore
.DS_Store is already on GitHub	It was previously committed	Add it to .gitignore, then use git rm --cached .DS_Store and commit
zsh: permission denied after entering a folder path	The path was entered as if it were a command	Use cd /path/to/folder
git add. fails	There is no space between add and .	Use git add .
git: 'orgin' does not appear to be a git repository	Typo in the remote name	Use origin
git commit shows automatic identity warning	Git inferred your name and email	Configure user.name and user.email

Git identity configuration

Set your preferred commit identity:

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_EMAIL"
```

This changes the identity used for future commits. It does not automatically rewrite the author identity of existing commits.

Remember the folder rule

To move into a folder:

```
cd /Users/gokulraj/Documents/VS/Python
```

Check your location:

```
pwd
```

A file must be inside the relevant repository directory to be added to that repository's Git history.

PART 8: High-Priority Interview Questions

1. What is Git?
Git is a free, distributed version control system that tracks changes to files and helps developers manage project history.

2. What is GitHub?
GitHub is an online platform for hosting Git repositories and collaborating on code.

3. What is a commit?
A commit is a saved snapshot of the project at a particular point in its history.

4. What is the difference between git add, git commit, and git push?
git add stages changes, git commit saves them in local history, and git push uploads commits to GitHub.

5. What is a branch?
A branch is an independent line of development that allows developers to work on features without immediately changing another branch.

6. What is the difference between git clone and git pull?
git clone creates a local copy of a remote repository. git pull retrieves and integrates remote changes into an existing local repository.

7. What is the difference between git fetch and git pull?

git fetch downloads remote updates without integrating them into the current branch. git pull fetches and then integrates them.

8. What is merging?

Merging combines changes from one branch into the currently checked-out branch.

9. What is .gitignore?

It specifies files and patterns Git should ignore when discovering untracked files.

10. What is detached HEAD?

Detached HEAD means Git is checked out directly at a commit rather than on a branch. It allows you to inspect or experiment with an earlier snapshot.

11. What is the difference between git revert and git reset?

git revert creates a new commit that reverses an earlier commit. git reset can move the current branch pointer and, depending on its options, alter the staging area or working tree.

12. How do you view previous versions?

Use git log to view history, git show <commit_id> to inspect a commit, and git diff to compare changes.

13. What is a remote repository?

A remote is a reference to another repository, commonly hosted on GitHub, used to exchange commits between local and remote repositories.

14. What is an upstream branch?

An upstream branch is the remote branch associated with a local branch for default push and pull operations.

15. What is a Pull Request?

A Pull Request proposes changes from one branch to another on GitHub so that the changes can be reviewed before merging.

PART 9: Final Command Cheat Sheet

Task	Command
Check Git installation	git --version
Initialize a repository	git init
Clone a repository	git clone <URL>
Check status	git status
Stage all changes	git add .
Stage a particular file	git add <file>
Commit changes	git commit -m "message"
Push changes	git push
First push of a branch	git push -u origin <branch>
Pull remote changes	git pull
Check remote URLs	git remote -v
Add a remote	git remote add origin <URL>
List local branches	git branch
List local and remote-tracking branches	git branch -a
Create a branch	git branch <name>
Create and switch to a branch	git switch -c <name>
Switch to an existing branch	git switch <name>

Task	Command
Merge a branch into the current branch	git merge <branch>
View short history	git log --oneline
Inspect a commit	git show <commit_id>
Compare unstaged changes	git diff
Compare staged changes	git diff --cached
Compare two commits	git diff <commit1> <commit2>
Check out a commit	git checkout <commit_id>
Unstage a file	git restore --staged <file>
Reverse a commit	git revert <commit_id>
Remove a file from tracking, keeping it locally	git rm --cached <file>
Change directory	cd <folder>
Show current directory	pwd
List hidden files	ls -la

The five commands to remember first

```
git status
git add .
git commit -m "message"
git push
git log --oneline
```

Understand what each command does before memorizing the syntax.

Your current learning status

You have practised the core Git workflow, version history, branch creation and switching, merging, pushing, pulling, upstream tracking, and .gitignore. Your next important topics are merge-conflict resolution, git revert versus git reset, and git stash.